UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



TRABAJO DE INVESTIGACIÓN FORMATIVA · PROYECTO FINAL

SGTE-UNSA: SISTEMA DE GESTIÓN DE TRÁMITES Y EXPEDIENTES ACADÉMICOS EN C++17 Y QT WIDGETS

Aplicación de Programación Orientada a Objetos, Patrones GoF, STL y Multithreading

Datos Académicos:

Asignatura: Tecnología de Objetos
Código: 1703240 · VI Ciclo · 2026-B
Grupo de Teoría: TEO B
Docente: Mg. Roni Guillermo Apaza Aceituno
Repositorio: `github.com/DenilsonCB/SGTE-UNSA`

Integrantes del Equipo:

- Choquehuanca Bedoya, Brayan Denilson
- Callo Ccagiavilca, Kevin Joel
- Ticona Saure, Jose Maria

Arequipa – Perú
2026

1. Introducción

La gestión de trámites administrativos estudiantiles en una escuela profesional involucra diferentes tipos de expedientes, reglas diferenciadas para el cálculo de tasas universitarias, niveles variables de prioridad y requisitos de trazabilidad documental. Cuando estos sistemas se construyen de manera procedimental o acoplando las reglas de cálculo dentro de los formularios de interfaz gráfica, el mantenimiento se vuelve costoso ante cualquier cambio normativo en el tarifario institucional.

El proyecto **SGTE-UNSA (Sistema de Gestión de Trámites y Expedientes Académicos)** resuelve esta problemática mediante una aplicación de escritorio desarrollada en C++17 y el framework Qt 6 Widgets. Su propósito académico consiste en integrar de forma cohesiva y verificable los principios de la asignatura Tecnología de Objetos: encapsulación, herencia, polimorfismo dinámico, punteros inteligentes, sobrecarga de operadores, manejo de excepciones, programación genérica con plantillas, algoritmos de la librería estándar STL con expresiones lambda, patrones de diseño GoF bajo arquitectura Modelo-Vista-Controlador (MVC) y ejecución concurrente mediante hilos de trabajo.

Como se detalla en la Tabla 1, cada módulo del código fuente ha sido proyectado para cubrir los ejes temáticos establecidos en el sílabo del curso.

Unidad / Tema del Sílabo	Archivo del Proyecto	Aplicación Concreta en SGTE-UNSA
POO, Herencia, Polimorfismo y `const` / `inline` / `friend`	`src/Tramite.h` `src/Tramite.cpp`	Clase base abstracta `Tramite` con subclases `TramiteMatricula`, `TramiteCartaPracticas` y `TramiteGradoBachiller`, además de sobrecarga de `operator<` y `operator==`.
Punteros Inteligentes y Memoria STL	`src/Tramite.cpp` `src/Repositorio.h`	Gestión automática del ciclo de vida de expedientes y estrategias mediante `std::shared_ptr<Tramite>` y `std::make_shared`.
Manejo de Errores y Excepciones	`src/Excepciones.h`	Jerarquía `ExcepcionTramite`, `ExcepcionValidacion` y `ExcepcionPersistencia` derivada de `std::runtime_error`.
Programación Genérica y Persistencia	`src/Repositorio.h`	Clase plantilla `Repositorio<T>` respaldada por `std::vector` y serialización hacia el archivo plano `tramites_unsa.csv`.
Expresiones Lambda y Algoritmos STL	`src/Repositorio.h` `src/MainWindow.cpp`	Consultas declarativas con `std::find_if`, `std::copy_if`, `std::count_if`, `std::accumulate`, `std::sort` y slots reactivos en Qt.
Multithreading y Objetos Función	`src/HiloProcesador.h`	Clase `HiloProcesador` derivada de `QThread` y functor `EvaluadorPrioridadFunctor` para auditar expedientes en segundo plano.
Patrones de Diseño GoF y MVC	`src/ControladorTramites.h` `src/EstrategiaTasa.h`	Arquitectura MVC combinada con **Singleton**, **Factory Method**, **Strategy** y **Decorator**.

Tabla 1: Trazabilidad entre los temas del sílabo de Tecnología de Objetos y los módulos de SGTE-UNSA.

La distribución mostrada en la Tabla 1 permite aislar las responsabilidades técnicas dentro del proyecto: las entidades del dominio permanecen libres de dependencias gráficas de Qt, facilitando su lectura, prueba y extensión.

2. Marco teórico

2.1 Polimorfismo dinámico, principios SOLID y gestión automática de memoria

El polimorfismo de subtipado permite que una referencia o puntero a una clase base abstracta invoque métodos sobrescritos (`override`) en sus clases derivadas mediante enlace dinámico (**dynamic binding**). Para cumplir con el Principio Abierto/Cerrado (OCP) de SOLID, las entidades deben estar abiertas para su extensión pero cerradas para su modificación directa. En C++ moderno, el uso de punteros inteligentes (`std::unique_ptr` y `std::shared_ptr`) de la cabecera `<memory>` automatiza la liberación determinista de recursos bajo el idioma RAII (**Resource Acquisition Is Initialization**), previniendo fugas de memoria al almacenar jerarquías polimórficas en contenedores estándar.

2.2 Programación genérica, contenedores STL frente a Qt y expresiones lambda

Las plantillas de clase (`template <typename T>`) habilitan el polimorfismo paramétrico en tiempo de compilación, permitiendo escribir estructuras de datos y repositorios independientes del tipo almacenado. En el desarrollo con C++ y Qt resulta fundamental contrastar y justificar la elección entre los contenedores de la librería estándar (`std::vector`) y los contenedores secuenciales de Qt (`QVector` y `QList`). En Qt 6, `QVector<T>` es un alias directo de `QList<T>`, el cual almacena sus elementos en un bloque contiguo de memoria con semántica **copy-on-write** (compartición implícita mediante conteo atómico de referencias) optimizada para el intercambio de datos con widgets gráficos. Por el contrario, `std::vector<T>` carece de sobrecarga por conteo atómico de copia y mantiene la capa de dominio 100% independiente del framework gráfico. Por esta razón, en SGTE-UNSA se seleccionó `std::vector<std::shared_ptr<T>>` dentro del modelo genérico `Repositorio<T>` y se reservaron las estructuras de Qt (`QStringList`, `QTableWidgetItem`) exclusivamente para la capa de vista (`MainWindow`).

Cuando las colecciones de la STL se procesan mediante algoritmos de `<algorithm>` y `<numeric>` (`std::find_if`, `std::copy_if`, `std::count_if`, `std::for_each`, `std::transform`, `std::accumulate` y `std::sort`) combinados con expresiones lambda (`[captura](parametros) { cuerpo }`), el compilador genera objetos clausura en línea que operan sin sobrecarga de indirección y mejoran la legibilidad frente a los bucles imperativos.

2.3 Patrones de diseño GoF y concurrencia en interfaces gráficas

Los patrones catalogados por Gamma et al. (GoF) se dividen en creacionales, estructurales y de comportamiento. Dentro de una aplicación con interfaz gráfica estructurada bajo el patrón Modelo-Vista-Controlador (MVC), la ejecución de tareas intensivas de entrada/salida sobre el hilo principal bloquea el bucle de eventos (**event loop**). El uso de hilos de trabajo (`QThread`) comunicados mediante señales asíncronas garantiza que la ventana mantenga su capacidad de respuesta mientras se procesan lotes de datos en segundo plano.

3. Diseño arquitectónico y patrones GoF

El sistema se organiza bajo el patrón arquitectónico MVC e incorpora cuatro patrones GoF seleccionados para resolver problemas específicos del dominio de trámites universitarios. Como se resume en la Tabla 2, se implementó al menos un representante de cada familia de patrones GoF.

Categoría	Patrón	Clases Participantes	Justificación Técnica en el Dominio
Arquitectónico	MVC	`Tramite` `MainWindow` `ControladorTramites`	Desacopla la ventana Qt de la lógica de validación, cálculo de tasas y persistencia en disco.
Creacional	Singleton	`ConfiguracionUNSA`	Centraliza en una instancia única el semestre académico activo (`2026-B`) y la ruta del archivo CSV.
Creacional	Factory Method	`TramiteFactory`	Encapsula la validación e instanciación polimórfica de trámites y estrategias a partir de cadenas de la GUI.
Comportamiento	Strategy	`IEstrategiaTasa` y 3 subclases	Permite intercambiar el algoritmo de liquidación de tasas (**Regular**, **Beca 50%** y **Exonerado**) en tiempo de ejecución.
Estructural	Decorator	`TramiteUrgenteDecorator`	Añade atención prioritaria (`Prioridad 0`) y recargo de urgencia (`+S/ 20.00`) envolviendo cualquier trámite existente.

Tabla 2: Patrones de diseño implementados en la arquitectura de SGTE-UNSA.

A partir de la estructura definida en la Tabla 2, la interacción entre el decorador estructural y las estrategias de comportamiento evita la explosión combinatoria de subclases: en lugar de codificar nueve clases distintas para combinar tres tipos de trámite con tres modalidades de tasa y urgencia, el diseño resuelve todas las combinaciones con tres subclases de `Tramite`, tres estrategias y un decorador.

En la Figura 1 se representa el diagrama de clases UML generado con PlantUML mostrando las relaciones entre los paquetes de Vista, Controlador y Modelo del sistema SGTE-UNSA.

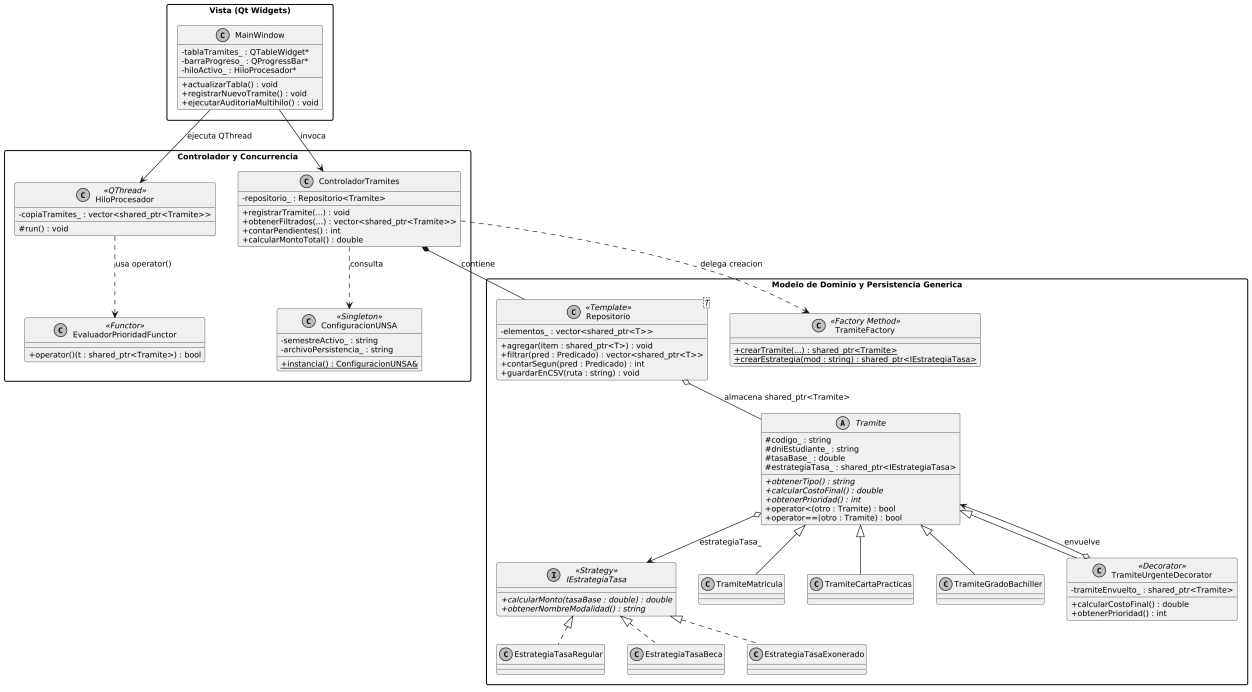


Figura 1: Diagrama de clases UML de la arquitectura MVC, jerarquía polimórfica de `Tramite` y patrones GoF.

Como se aprecia en la Figura 1, la clase abstracta `Tramite` mantiene una relación de agregación polimórfica con la interfaz `IEstrategiaTasa` y es extendida tanto por las tres subclases de dominio como por el decorador

`TramiteUrgenteDecorator`, mientras que `MainWindow` se comunica con `ControladorTramites` y lanza el hilo `HiloProcesador`.

4. Implementación

4.1 Jerarquía polimórfica, sobrecarga de operadores y patrones en `Tramite.h`

La clase base abstracta `Tramite` encapsula los atributos comunes del expediente y define la interfaz polimórfica junto con la sobrecarga de los operadores relacionales `operator<` y `operator==`. A continuación se presenta el fragmento correspondiente de `src/Tramite.h` (líneas 35 a 87).

```
// src/Tramite.h (Líneas 35–87)
 35 class Tramite {
 36 protected:
 37     std::string codigo_;
 38     std::string dniEstudiante_;
 39     std::string nombreEstudiante_;
 40     std::string estado_;
 41     double tasaBase_;
 42     std::shared_ptr<IEstrategiaTasa> estrategiaTasa_;
 43
 44 public:
 45     Tramite(std::string codigo,
 46             std::string dni,
 47             std::string nombre,
 48             double tasaBase,
 49             std::shared_ptr<IEstrategiaTasa> estrategia,
 50             std::string estado = "Pendiente");
 51
 52     virtual ~Tramite() = default;
 53
 54     virtual std::string obtenerTipo() const = 0;
 55     virtual double calcularCostoFinal() const = 0;
 56     virtual int obtenerPrioridad() const = 0; // 0 = Urgente, 1 = Alta, 2 = Media, 3 = Normal
 57     virtual bool esUrgente() const { return false; }
 58
 59     inline const std::string& getCodigo() const { return codigo_; }
 60     inline const std::string& getDni() const { return dniEstudiante_; }
 61     inline const std::string& getNombre() const { return nombreEstudiante_; }
 62     inline const std::string& getEstado() const { return estado_; }
 63     inline double getTasaBase() const { return tasaBase_; }
 64     inline std::string getModalidadTasa() const {
 65         return estrategiaTasa_ ? estrategiaTasa_->obtenerNombreModalidad() : "Regular";
 66     }
 67
 68     void setEstado(const std::string& nuevoEstado) { estado_ = nuevoEstado; }
 69     void setEstrategiaTasa(std::shared_ptr<IEstrategiaTasa> nuevaEstrategia) {
 70         estrategiaTasa_ = std::move(nuevaEstrategia);
 71     }
 72
 73     virtual std::string serializarCSV() const;
 74
 75     bool operator<(const Tramite& otro) const {
 76         if (this->obtenerPrioridad() != otro.obtenerPrioridad()) {
 77             return this->obtenerPrioridad() < otro.obtenerPrioridad();
 78         }
 79         return this->codigo_ < otro.codigo_;
 80     }
 81
 82     bool operator==(const Tramite& otro) const {
 83         return this->codigo_ == otro.codigo_;
 84     }
 85
 86     friend std::ostream& operator<<(std::ostream& os, const Tramite& t);
 87 };
```

En el bloque anterior se observa que `operator<` compara en primera instancia el resultado polimórfico de `obtenerPrioridad()`, donde `0` representa prioridad urgente y `3` prioridad normal, utilizando el código lexicográfico como criterio de desempate. Por su parte, la declaración `friend std::ostream& operator<<` permite exportar el expediente formateado directamente hacia flujos de archivo durante la auditoría.

4.2 Repositorio genérico, algoritmos STL y funciones lambda en `Repositorio.h`

Para desacoplar el almacenamiento respecto al tipo de entidad, se implementó la plantilla `Repositorio<T>` en `src/Repositorio.h`. El siguiente fragmento (líneas 52 a 101) muestra la integración de plantillas de método con predicados lambda y algoritmos de la STL.

```
// src/Repositorio.h (Líneas 52–101)

52      template <typename Predicado>
53      std::vector<std::shared_ptr<T>> filtrar(Predicado pred) const {
54          std::vector<std::shared_ptr<T>> resultado;
55          std::copy_if(elementos_.begin(), elementos_.end(), std::back_inserter(resultado), pred);
56          return resultado;
57      }
58
59      template <typename Predicado>
60      int contarSegun(Predicado pred) const {
61          return static_cast<int>(std::count_if(elementos_.begin(), elementos_.end(), pred));
62      }
63
64      double calcularMontoTotal() const {
65          return std::accumulate(elementos_.begin(), elementos_.end(), 0.0,
66              [](double acumulado, const std::shared_ptr<T>& item) {
67                  return acumulado + item->calcularCostoFinal();
68              });
69      }
70
71      void ordenarPorPrioridad() {
72          std::sort(elementos_.begin(), elementos_.end(),
73              [](const std::shared_ptr<T>& a, const std::shared_ptr<T>& b) {
74                  return *a < *b;
75              });
76      }
77
78      const std::vector<std::shared_ptr<T>>& obtenerTodos() const {
79          return elementos_;
80      }
81
82      std::size_t tamano() const {
83          return elementos_.size();
84      }
85
86      void limpiar() {
87          elementos_.clear();
88      }
89
90      void guardarEnCSV(const std::string& rutaArchivo) const {
91          std::ofstream archivo(rutaArchivo, std::ios::out | std::ios::trunc);
92          if (!archivo.is_open()) {
93              throw ExcepcionPersistencia(rutaArchivo);
94          }
95          archivo << "CODIGO;TIPO;DNI;ESTUDIANTE;MODALIDAD;URGENTE;COSTO_FINAL;ESTADO\n";
96          std::for_each(elementos_.begin(), elementos_.end(),
97              [&archivo](const std::shared_ptr<T>& item) {
98                  archivo << item->serializarCSV() << "\n";
99              });
100     }
101 };
```

El diseño de `Repositorio<T>` evita exponer los iteradores internos de `std::vector<std::shared_ptr<T>>`. En su lugar, expone operaciones funcionales de alto nivel: `filtrar` emplea `std::copy_if` con `std::back_inserter`, `contarSegun` envuelve `std::count_if`, `calcularMontoTotal` reduce la colección me-

diante `std::accumulate` y `guardarEnCSV` recorre los elementos con `std::for_each` capturando el flujo de archivo por referencia (`[&archivo]`).

4.3 Procesamiento concurrente con `QThread` y objeto función en `HiloProcesador.h`

El módulo de concurrencia emplea un objeto función (**functor**) y una subclase de `QThread` para auditar la lista de expedientes y generar un archivo de liquidación en disco sin bloquear el hilo de interfaz gráfica. A continuación se muestra la implementación en `src/HiloProcesador.h` (líneas 12 a 73).

```
// src/HiloProcesador.h (Líneas 12–73)

12 struct EvaluadorPrioridadFunctor {
13     bool operator()(const std::shared_ptr<Tramite>& t) const {
14         return t->obtenerPrioridad() <= 1 && t->getEstado() != "Atendido";
15     }
16 };
17
18 class HiloProcesador : public QThread {
19     Q_OBJECT
20
21 private:
22     std::vector<std::shared_ptr<Tramite>> copiaTramites_;
23     std::string rutaReporte_;
24
25 public:
26     HiloProcesador(std::vector<std::shared_ptr<Tramite>> tramites,
27                    std::string rutaReporte,
28                    QObject* parent = nullptr)
29         : QThread(parent),
30           copiaTramites_(std::move(tramites)),
31           rutaReporte_(std::move(rutaReporte)) {}
32
33 signals:
34     void progresoActualizado(int porcentaje, const QString& detalle);
35     void procesamientoFinalizado(const QString& resumenFinal);
36
37 protected:
38     void run() override {
39         std::ofstream reporte(rutaReporte_, std::ios::out | std::ios::trunc);
40         reporte << "================================================================\n";
41         reporte << "   REPORTE DE AUDITORIA Y LIQUIDACION DE EXPEDIENTES - UNSA    \n";
42         reporte << "   Semestre Academico: " << ConfiguracionUNSA::instancia().getSemestreActivo() << "\n";
43         reporte << "================================================================\n\n";
44
45         int total = static_cast<int>(copiaTramites_.size());
46         int prioritariosPendientes = 0;
47         double recaudacionTotal = 0.0;
48         EvaluadorPrioridadFunctor esPrioritario;
49
50         if (total == 0) {
51             emit progresoActualizado(100, "Sin expedientes para procesar.");
52             emit procesamientoFinalizado("No hay expedientes registrados.");
53             return;
54         }
55
56         for (int i = 0; i < total; ++i) {
57             const auto& t = copiaTramites_[i];
58             if (esPrioritario(t)) {
59                 ++prioritariosPendientes;
60             }
61             recaudacionTotal += t->calcularCostoFinal();
62
63             reporte << (i + 1) << ". " << *t << "\n";
64
65             QThread::msleep(180);
66
67             int porcentaje = ((i + 1) * 100) / total;
68             QString msg = QString("Verificando expediente %1 (%2/%3)...")
```

```
                            .arg(QString::fromStdString(t->getCodigo()))
                            .arg(i + 1)
                            .arg(total);
            emit progresoActualizado(porcentaje, msg);
        }
```

Para evitar condiciones de carrera sobre el vector de la interfaz principal, el constructor de `HiloProcesador` recibe por valor una copia ligera del vector de `std::shared_ptr<Tramite>`. Durante la ejecución de `run()`, el hilo secundario invoca el operador sobrecargado `operator()` de `EvaluadorPrioridadFunctor` y notifica el avance a la ventana principal mediante la señal `progresoActualizado`, la cual es encolada de forma segura por el sistema de meta-objetos de Qt.

5. Ejecución del código y explicación de resultados

El proyecto fue compilado utilizando el estándar C++17 con el compilador MinGW 13.1 de 64 bits y Qt 6.12.0 bajo las banderas `-Wall -Wextra`. En la Figura 2 se observa la ventana principal de **SGTE-UNSA** tras cargar los expedientes iniciales desde el archivo `tramites_unsa.csv`.

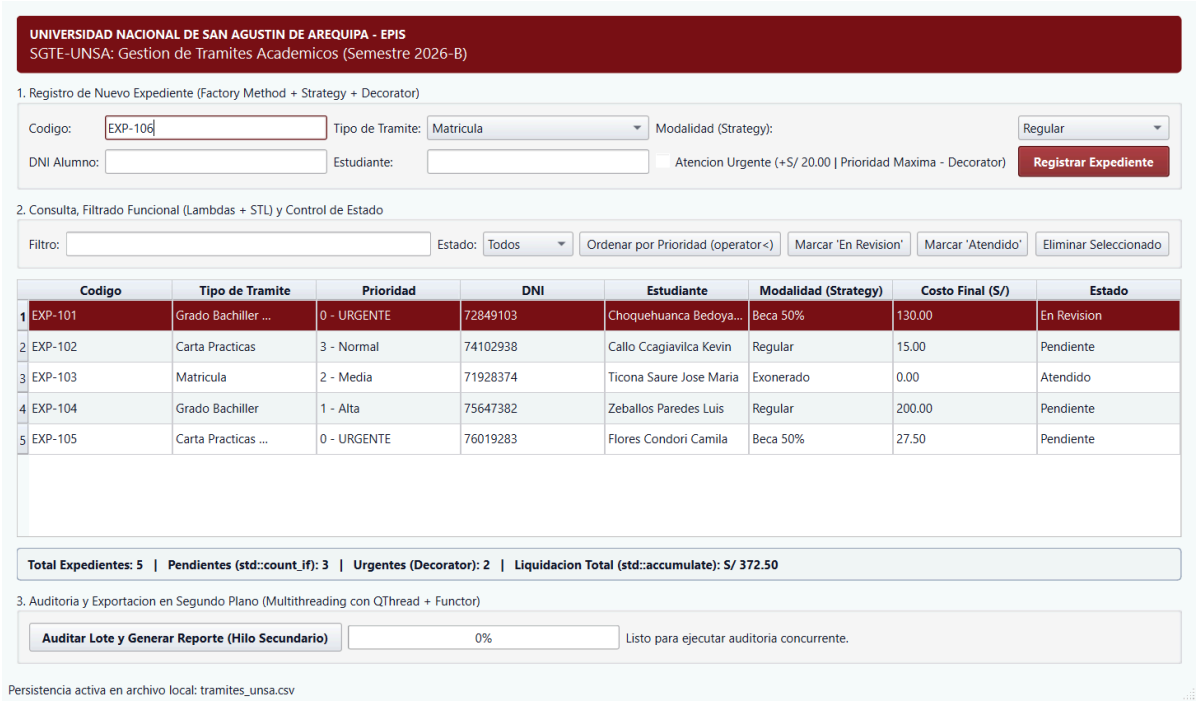


Figura 2: Interfaz gráfica principal de SGTE-UNSA mostrando el registro polimórfico, el filtrado funcional y las métricas calculadas con algoritmos STL.

En la Figura 2 se comprueba el cálculo polimórfico de tasas y la aplicación de los patrones **Strategy** y **Decorator**:

- El expediente `EXP-101` (**Grado Bachiller**, tasa base S/ 180.00 + S/ 20.00 de diploma) bajo modalidad **Beca 50%** reduce su base a S/ 90.00 (+ S/ 20.00 = S/ 110.00) y, al estar envuelto por `TramiteUrgenteDecorator`, suma S/ 20.00 adicionales alcanzando un costo final de `S/ 130.00` con prioridad `0 - URGENTE`.
- El expediente `EXP-103` (**Matrícula**) con estrategia **Exonerado** registra un costo final de `S/ 0.00`.
- En la barra inferior de métricas se comprueba que `std::count_if` identifica 3 expedientes pendientes y 2 urgentes, mientras que `std::accumulate` totaliza `S/ 372.50`.

Posteriormente se ejecutó el ordenamiento polimórfico mediante `std::sort` y el hilo de auditoría. La Figura 3 presenta el estado de la interfaz tras ordenar los expedientes por nivel de prioridad (`0 - URGENTE`, `1 - Alta`, `2 - Media`, `3 - Normal`) y completar al 100% la auditoría concurrente con `QThread`.

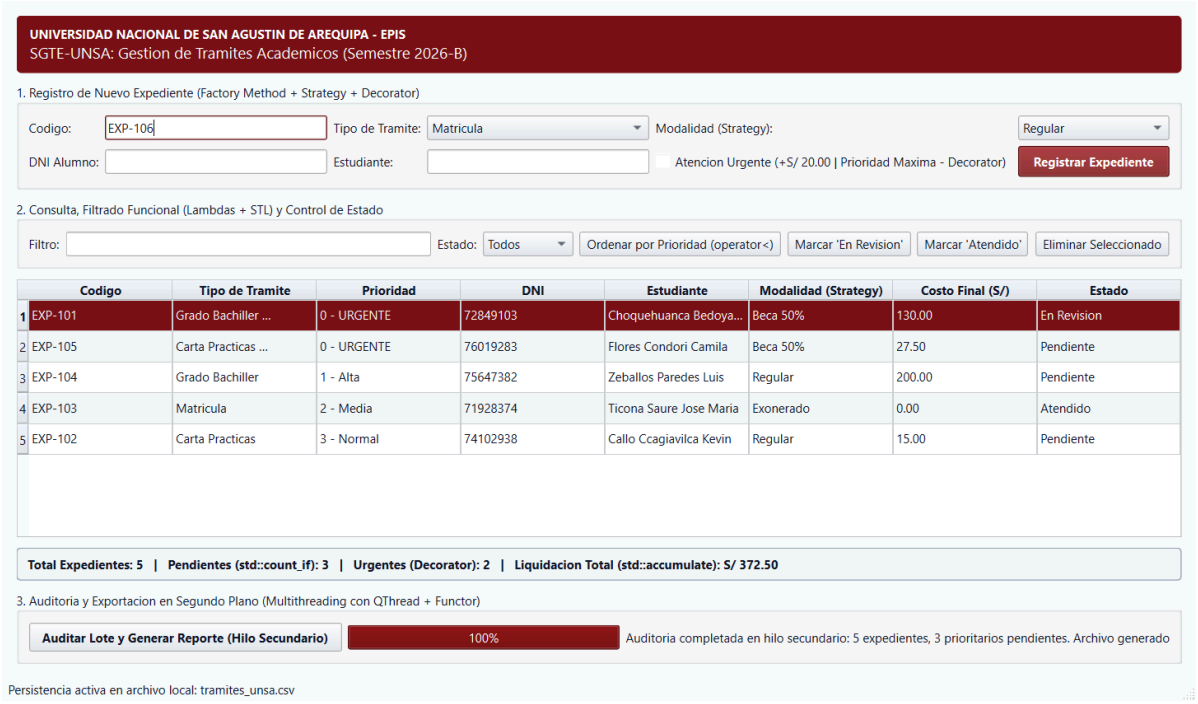


Figura 3: Ordenamiento por `operator<` y culminación de la auditoría en segundo plano mediante `QThread` y `EvaluadorPrioridadFunctor`.

Como muestra la Figura 3, los dos expedientes decorados como urgentes (`EXP-101` y `EXP-105`) se posicionan en las primeras filas de la tabla gracias a la sobrecarga de `operator<`, seguidos por el trámite de **Grado Bachiller** (`1 - Alta`), **Matrícula** (`2 - Media`) y **Carta Prácticas** (`3 - Normal`). Asimismo, la barra `QProgressBar` alcanza el `100%` y reporta la generación del archivo `reporte_auditoria.txt` con 3 alertas de prioridad alta o urgente pendientes de atención.

6. Conclusiones

El desarrollo del sistema SGTE-UNSA permitió integrar los pilares de la programación orientada a objetos y los paradigmas modernos de C++17 dentro de una arquitectura modular Modelo-Vista-Controlador en Qt 6. La definición de la clase abstracta `Tramite` y sus tres subclases demostró cómo el polimorfismo dinámico y el uso de punteros inteligentes `std::shared_ptr` simplifican el manejo de colecciones heterogéneas sin fugas de memoria.

La incorporación conjunta de los patrones creacionales (Singleton y Factory Method), de comportamiento (Strategy) y estructurales (Decorator) permitió desacoplar la creación de objetos y el cálculo de tasas universitarias respecto a la interfaz gráfica. En particular, la combinación de `IEstrategiaTasa` con `TramiteUrgenteDecorator` evitó duplicar clases para representar becas, exoneraciones y recargos de urgencia, satisfaciendo el principio Abierto/Cerrado.

Finalmente, la implementación de la plantilla `Repositorio<T>` apoyada en expresiones lambda, algoritmos de la librería estándar STL (`std::find_if`, `std::copy_if`, `std::count_if`, `std::accumulate`, `std::sort`) y un hilo secundario basado en `QThread` y objetos función demostró que la combinación de programación genérica, funcional y concurrente produce código conciso, expresivo y capaz de ejecutar operaciones de entrada/salida en segundo plano sin degradar la experiencia interactiva del usuario.

7. Referencias bibliográficas

1. B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
2. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1995.
3. J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2017.
4. B. Meyer, «Applying “design by contract”,» *Computer*, vol. 25, no. 10, pp. 40–51, Oct. 1992.
5. The Qt Company, *Qt 6.12 Documentation: Qt Widgets and QThread Reference*, 2026. [Online]. Available: https://doc.qt.io/qt-6/